

SOTWISE x GSS — mapeamento das bibliotecas e ações

Confronto entre o schema de produção do SOTWISE (Supabase AGK) e o app `apps.core` do GSS · 11 ago 2026

Decisão de base: o GSS passa a ser o dono das bibliotecas; o SOTWISE apenas consome, read-only, por pull agendado. O que estiver desnormalizado do nosso lado será corrigido.

Medido em produção: `contacts`, `carrier_agents` e os agentes Brazil/China somam **8 referências** em todo o checklist — existem, mas ninguém preenche. Não são bloqueadores.

NADA A FAZER NORMALIZAR AQUI PEDIR AO GSS MANTER LOCAL DECIDIR

1. Bibliotecas que vão sincronizar — têm os dois lados

SOT	GSS	AÇÃO
<code>countries</code> 3	Country	NADA Opcional: trazer o <code>iso_code</code> .
<code>cities</code> 51	City	NADA Eles têm <code>Province</code> acima da cidade; ignorar não quebra nada.
<code>pols</code> 74 + <code>Pods</code> 26	Port	NORMALIZAR 74 → 23 POLs reais (as 74 linhas são porto × cidade de origem repetido). Repontar 4.768 etapas de checklist. PEDIR discriminador POL/POD — <code>Port</code> não diz qual é qual.
<code>factories</code> 752	SupplierCategory	NORMALIZAR Mesclar 92 linhas → 40 fornecedores (mesmo fornecedor repetido por categoria). Repontar 2.023 linhas de <code>order_factory_category</code> . A junção <code>category_factories</code> passa a vir do <code>SupplierCategory</code> .
<code>categories</code> 115	Category	LIMPAR 9 nomes vazios e 4 duplicados. PEDIR as colunas do <code>Category</code> — está em outro app e o ERD não mostra.
<code>clients</code> 115	Customer	DECIDIR a regra do <code>Company.name</code> (não há FK direta Customer→Company). Hoje 114 de 115 sem país — passa a vir do GSS.
<code>agents</code> 144	Agent	PEDIR o <code>country</code> . O nosso <code>location</code> (brazil/china) deriva dele — e sem ele os dropdowns Agent Brazil/China ficam vazios.
<code>carriers</code> 26	Carrier	NADA Perdemos o telefone (eles têm <code>address</code> , não <code>phone</code>).
<code>exporters</code> 4	Exporter	DECIDIR o <code>acronym</code> (AGK, ZC, ZY, ZT): campo novo no GSS ou relaxar o NOT NULL e manter local. Único caso que quebra insert .
<code>business_units</code> 7	BusinessUnit	NADA LOCAL Ícone segue no nosso Storage (o <code>icon</code> deles é arquivo, não viaja no sync).
<code>order_types</code> 5	OrderType	NADA LOCAL <code>color</code> e ícone seguem nossos — não existem lá.
<code>shipment_models</code> 6	— vão criar	AGUARDAR Entra no sync quando existir no GSS.

2. Só no SOTWISE — sem fonte no GSS

SOT	GSS	AÇÃO
<code>contacts</code> 288	não existe	CONGELAR Read-only, fora do sync. Uso real: 3 referências — não vale pedir <code>Contact</code> ao GSS.
<code>agent_contacts</code>	não existe	CONGELAR Vai junto com <code>contacts</code> .
<code>carrier_agents</code>	não existe	CONGELAR 3 usos. Base do filtro “Carrier agent” do Pre-loading.
<code>exporters.acronym</code>	—	DECIDIR NOT NULL sem origem — falha o insert. Campo no GSS ou virar dado local.
<code>agents.location</code>	Agent sem país	PEDIR Derivar do <code>country</code> que vamos pedir. Sem isso, feature quebrada.
<code>phone_number</code> <code>agents/carriers</code>	—	DECIDIR Aceitar a perda ou pedir o campo.
<code>color</code> e ícones	—	LOCAL Ficam nossos, chaveados pelo <code>gss_id</code> . Única exceção ao read-only.
<code>email_na</code>	—	DERIVAR E-mail vazio no GSS → <code>email_na = true</code> .
<code>clients</code> sem flag inativo	Customer sem active	PEDIR Sem flag, cliente nunca sai da lista.
<code>deleted_at</code> todas	não existe	NOSSO LADO Varredura completa periódica detecta o que sumiu. Nos volumes atuais é barato e não depende do GSS mudar nada. <code>Supplier.is_obsolete</code> já serve para as fábricas.

3. Só no GSS — sem correspondência nossa

SOT	GSS	AÇÃO
—	Company	DECISÃO GRANDE Trazer como tabela nossa (normaliza de vez: uma empresa pode ser fábrica e exportadora) ou só puxar o <code>name</code> . Tem <code>company_id</code> , endereço, logo, contato, <code>short_name</code> .
—	Supplier	USAR <code>is_obsolete</code> + <code>obsolete_justification</code> é exatamente o nosso soft delete das fábricas, com o motivo de brinde.
—	Province	DECIDIR Criar a tabela ou ignorar (perda pequena).
—	PaymentCondition	PEDIR as colunas (está em outro app). Candidato a trazer — <code>Customer</code> já aponta para ela.
—	CustomerConsignee CustomerImporter	AVALIAR Consignatário e importador por cliente. Dado de negócio real que não existe em lugar nenhum do SOTWISE — nem em orders, nem em pre-loadings, nem em shipments.
—	Currency	IGNORAR Não temos nada de moeda hoje.
—	Family	PERGUNTAR O que é, e o que significa <code>requires_model_application</code> .
—	SalesRepresentative + carteira e comissão	FORA DO ESCOPO Domínio comercial que o SOTWISE não modela.
—	SupplierCategory .city / .code	DECIDIR Se viram colunas nossas. Hoje não temos onde guardar.
—	SystemRelease SystemChangelog	IGNORAR Versionamento do app deles.

O trabalho de normalização são 2 itens: dedupe de `pols` (4.768 etapas de checklist) e merge de `factories` (2.023 linhas de `order_factory_category`). Os dois são determinísticos — a tabela `order_factory_category` já carrega o par (fábrica, categoria), então o destino de cada linha é conhecido. Sobram ~320 linhas para decisão manual: 215 usam um par que a biblioteca não conhece e 107 apontam para fábricas sem categoria.

Duas premissas do GSS que precisam de resposta antes do puller: a PK é `BigAutoField` (inteira), então o pareamento inicial exige um campo `sotwise_id` lá — pareamento por nome **não funciona** em `pols` e `factories`, onde o nome não identifica a linha. E `Supplier` não tem `updated_at`: renomear a empresa no GSS não levantaria a mudança — a view precisa expor `greatest(supplier_category.updated_at, company.updated_at)`.